

Pytorch Bu facebook tomonidan yaratilgan open-source deep learning kutubxonasidir.

=> Automatic Differentiation (autograd) `backward()` gradientni avtomatik hisoblaydi.

=> Tensorlarni oson GPU ga o'tkazish m-m. `tensor.to("cuda")` trainni tezlashtiradi.

Modullar

1. `torch` => asosiy tensor operatsiyalari
2. `torch.nn` => Neural networks layers (Linear, relu, ...)
3. `torch.optim` => Optimizatorlar (Adam, SGD...)
4. `torch.utils.data` => Dataset, Data Loader. ma'lumotlar bilan ishlash.
5. `torch.autograd` => Gradientlarni avtomatik hisoblash.

Pytorch is a foundation for

- CNN (Computer Vision)
- Transformers (LLM)
- Multimodal AI
- Reinforcement Learning
- AI Agents, backend models.

Optimizer Weightlorni gradient yordamida update qilish uchun usul.

DL Training:

- Forward pass $\rightarrow$ prediction
- Loss calculation
- Backward $\rightarrow$ Gradients
- optimizer.step() $\rightarrow$ update weights

loss.backward() $\Rightarrow$ gradientni hisoblaydi

.backward() $\Rightarrow$ weightsni o'zgartirmaydi.

u foyzat g'ayri to'g'ri yurish kerakligini ko'rsatadi.

Optimizer ayvon shu gradient asosida weights ni yangilaydi. opt.step()

Turlari: SGD

SGD + Momentum

ADAM

ADAMW

RMSprop

Adagrad

Eng oddiy: SGD (Stochastic Gradient descent)

$$w_{\text{new}} = w_{\text{old}} - lr \cdot \text{gradient}$$

kamchiligi $\Rightarrow$ barcha weights uchun shu learning rate ishlatiladi.

Adaptive moment Estimation

Adam $\Rightarrow$ alohida learning rate.

Momentum (oldingi qadamlorni taqab qoladi)

Optimizer nima?

Gradientlar asosida modeling
weights ni yangilaydigan algoritim.

Nga korak?

backward() - lagat gradient
hisoblaydi, uni amolga oshiruvchi
bu optimizer

Euf kor, ishlatiladigan:

Adam - tez va bog'aror.

Euf muhim parametri:

lr (learning rate) - qadamlar kattaligi

Batch size

opt. step() - cheklangan